A Simple Average-Case Analysis of The Reverse Factor Algorithm

Kuei-Hao Chen[1], Guan-Shieng Huang[1] and Richard Chia-Tung Lee[2,*]

[1]Department of Computer Science and Information Engineering,
National Chi Nan University, Puli, Nantou, Taiwan.
{s95321902, shieng}@ncnu.edu.tw
[2]Department of Computer Science,
National Tsing Hua University, Hsinchu, Taiwan.
rctlee@ncnu.edu.tw

Abstract

We analyzed the average performance of the reverse factor (RF) algorithm for exact string matching. The analysis is based on the assumption that the text is very long as compared to the length of the pattern, and each symbol in the text is drawn uniformly from a random source with r symbols. We show that the average number of character comparisons for the RF algorithm to find all occurrences of any given pattern in a random text is $O\left(\frac{n \log_r m}{m}\right)$*, and this result coincides with the previous result provided by Crochemore et al. in [4]. However, our analysis is much simpler and uses only elementary techniques in probability theory. We also conducted a series of experiments for the RF algorithm and got several useful statistics from random texts and human chromosomes.*

1 Introduction

String matching is an important topic in computer science because it is the fundamental building blocks for many applications. For example, it appears in text editors, search engines and electronic dictionaries. The simplest brute-force algorithm has the worst-case time complexity $O(nm)$ where n and m are the lengths of a text and a pattern, respectively. This is possibly the only way for an unskillful programmer to implement the function of string matching. Even though it has poor performance in worst-case, this brute-force algorithm has competitive performance to many sophisticated string matching algorithms in average case (more details will be explained in the next paragraph). This phenomenon reveals a distinction between worst-case and average-case analyses, and the application of each principle has its own limitation and appropriateness. Thus, one should carefully interpret the meanings of results analyzed according to different principles and avoid overstating their implications.

There are two types of settings for the analysis of average-case complexity of an exact string matching algorithm. The first one considers the average number of character comparisons performed by an individual algorithm in order to locate the first occurrence of the pattern in the text if it does exist. We use the notation $\overline{C}_{\texttt{first\_match}}$ to denote this number under this setting. The second one tries to evaluate the average number of character comparisons needed in order to find all occurrences of the pattern in the text. Let us denote this number as $\overline{C}_{n,m}$.

The KMP algorithm [8] is one of the most famous string matching algorithms appeared in many text books. Barth in [1] showed that $\overline{C}_{\texttt{first\_match}}$ for the KMP algorithm is $r^m + \frac{1}{r-1}r^{m-1} + r - \frac{r}{r-1}$ where r is the size of the alphabet and for the brute-force algorithm is $\frac{r^{m+1}}{r-1} - \frac{r}{r-1}$. The ratio of these two numbers is approximately $1 - \frac{1}{r} + \frac{1}{r^2} + \frac{r-1}{r^m} = O(1)$, which indicates that there is no too much difference on the average performance between these two algorithms in order to find the first occurrence of a random pattern in a random text. Baeza-Yates in [2] showed that $\overline{C}_{n,m}$ for the KMP algorithm satisfied

$$\frac{\overline{C}_{n,m}}{n} \leq 2 - \frac{1}{r} + O\left(\frac{1}{n}\right) < 2 \quad \text{for } r \leq |\log_{\phi} m|$$

*To whom correspondence should be addressed. Email:rctlee@ncnu.edu.tw

and

$$\frac{\overline{C}_{n,m}}{n} \leq 1 + \frac{1}{r} + \frac{\lceil \log_{\emptyset} m \rceil - 1}{r^3} + O\left(\frac{\log^2 m}{r^4}\right)$$
$$\text{for } r > |\log_{\emptyset} m|, \text{ where } \emptyset = \tfrac{1+\sqrt{5}}{2}.$$

As for the brute-force algorithm, it follows that $\overline{C}_{n,m} = \frac{r}{r-1}\left(1 - \frac{1}{r^m}\right)(n-m+1) + O(1)$. Baeza-Yates also showed that $\overline{C}_{n,m}$ for the Horspool algorithm [6] satisfied $\frac{\overline{C}_{n,m}}{n} = \frac{1}{m} + \frac{m+1}{2mr} + O\left(r^{-2}\right)$.

Another famous algorithm for exact string matching is the BM algorithm [3], whose $\overline{C}_{n,m}$ value was analyzed by Knuth *et al.* in [8], which is $O\left(\frac{n \log_r m}{m}\right)$. Knuth *et al.* also conjectured that this bound is tight, and it was affirmed by Yao in [10]. In fact, Yao proved a more general result. He showed that this average-case lower bound $\Omega\left(\frac{n \log_r m}{m}\right)$ holds for any exacting string matching algorithm.

The reverse factor algorithm (shortened as the RF algorithm), which was proposed by Crochemore *et al.* [4], is based on the following idea. It opens a sliding window of width equal to m, moves this window from left to right, and scan characters in each window from right to left. While scanning a window, it maintains a data structure that keeps the information: the longest match of a prefix of the pattern to a suffix of the window. A suffix tree or directed acyclic graph building for the pattern is a good candidate to serve this purpose. Its average complexity $\overline{C}_{n,m}$ was also analyzed in [4], which is $O\left(\frac{n \log_r m}{m}\right)$. However, implementation of a suffix tree is not an easy task, and furthermore, its performance is not beneficial when the size of the pattern is not large enough. Therefore, Navarro and Raffinot in [9] proposed another algorithm with similar basic idea to the RF algorithm but using bit-parallel approach instead of employing a suffix tree.

The organization of this paper is as follows. Section 2 describes the basic idea of the RF algorithm. An analysis of the average-time complexity for the RF algorithm is given in Sect. 3. Then the results for two sets of experiments are shown in Sect. 4. And finally, Sect. 5 concludes this work.

2 Preliminaries

In this section, we briefly introduce the basic idea of the RF algorithm [4]. In the RF algorithm, it opens a sliding window of length m on the text,

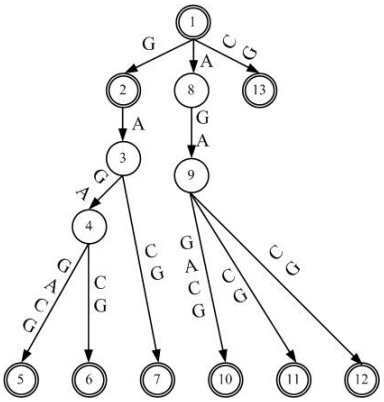

Figure 1: A reverse suffix tree of P.

and in each attempt it tries to find a longest suffix of the window which is equal to a prefix of the pattern. If the whole pattern is matched with the window, then an occurrence of the pattern is found, and it locates the second longest suffix-to-prefix match. Subsequently, it moves the window to the right such that these two matched substrings are aligned, and the next attempt starts. The question is on how to implement this basic idea. The RF algorithm is based on constructing a suffix tree for the pattern during the preprocessing phase. Given a pattern P and a window W, we want to find the longest suffix of W which is equal to a prefix of P. We can first construct the reverse suffix tree of P, which is indeed a suffix tree for the reverse string of P. For example, let P=GCAGAGAG. We can construct a reverse suffix tree of P in Fig. 1. In Fig. 1, all states are re-numbered by the depth-first search. The terminal states represent the reversed prefixes of P.

We feed W to the suffix tree from right to left, followed from the root down to leaves in the tree. The last reached prefix is the longest one. There are two cases:

1. We reach the beginning of the window. We report the occurrence, and we shift the window exactly as the last terminal node.

2. We fail to find a factor of P as the currently scanned suffix of this window. Then we find the last reached terminal node, and use the string spelled out from this node to the root as the matched prefix of P, and move the pattern right so that the matched prefix and suffix are aligned.

Using above method, we can also find the longest suffix of a window in text which is equal to a prefix of the pattern.

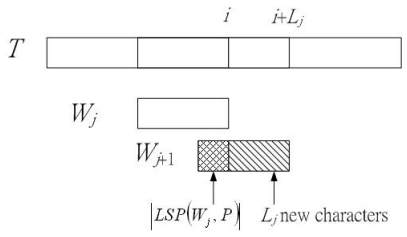

Figure 2: After the jth attempt, we have L_j new characters.

3 Materials and Method

In Crochemore analysis, let L_j be the length of the shift in the jth attempt of RF algorithm and let S_j be the length of substring of the pattern that is matched in this attempt. Let $LSP(W,P)$ denote a longest suffix of the window of T which is equal to a prefix of P such that $S_j \geq LSP(W,P)$ They use S_j to analyze the average case of RF algorithm finally.

In fact, the main idea of RF algorithm is to find out $LSP(W,P)$, but not S_j. Therefore, we propose a simple to prove the expected number of comparisons of the RF algorithm from another point.

Let $\overline{P}$ denote the reverse string of pattern P. In the RF algorithm, it builds a suffix tree of $\overline{P}$ and inspects each window W of text T by using the suffix tree [4]. Therefore, we are given a window W of text T and a pattern P, $W = w_1 w_2 \cdots w_m$, $P = p_1 p_2 \cdots p_m$. Let L be the length of the shift, $L \in \{1, 2, \cdots, m\}$. Then $m = L + |LSP(W,P)|$. We can analyze the average value of $|LSP(W,P)|$, called $\overline{C}_{|LSP(W,P)|,m}$.

Let W_j be the string in windows W for jth attempt. When W_j moves to W_{j+1} , suppose the end position moves from position i to position $i + L_j$. Then it has L_j new characters and the length of the overlap between W_j and W_{j+1} is $|LSP(W_j, P)|$, as shown in Fig. 2.

Let A_ℓ be the expectation of character comparisons for W_{j+1} under the condition that $L_j = \ell$. Then

$$A_\ell = \sum_{k=0}^{m} \Pr\left((|LSP(W_{j+1},P)| = k)\,|L_j = \ell\right) \times k \ . \quad (1)$$

We have

$$\sum_{\ell=0}^{m} \Pr(L_j = \ell) \times A_\ell$$

$$= \sum_{\ell=0}^{m} \Pr(L_j = \ell) \times$$

$$\sum_{k=0}^{m} \Pr\left((|LSP(W_{j+1},P)| = k)\,|L_j = \ell\right) \times k$$

$$= \overline{C}_{|LSP(W_{j+1},P)|,m} \quad (2)$$

Furthermore, we have

$$\Pr(L_j = m) \geq \Pr(L_j = m-1) \geq \cdots \geq \Pr(L_j = 1) \quad (3)$$

since $\Pr(|LSP(W_{j+1},P)| = k)$ decreases as k increases and $m = L_j + |LSP(W_j,P)|$.

For calculating A_ℓ, we first consider the case that shift is equal to m. In the next lemma, we show that $A_m = \frac{r}{(r-1)^2}$ where r is the alphabet size of P.

Lemma 1 *Let m be the length of P and r be the alphabet size of P. Then A_m is bounded $\frac{r}{(r-1)^2}$.*

Proof. Note that $LSP(W,P)$ denotes the longest suffix of W in T which is equal to a prefix of P. Then we have

$$\begin{aligned}
\Pr(|LSP(W,P)| = 1) &= \frac{1}{r}, \\
\Pr(|LSP(W,P)| = 2) &= \frac{1}{r^2}, \\
&\vdots \\
\Pr(|LSP(W,P)| = m) &= \frac{1}{r^m}
\end{aligned}$$

since there are m random symbols in W for calculating A_m. Thus A_m can be found by the following derivations from Equation (1).

$$\begin{aligned}
A_m &= 1 \times \frac{1}{r} \times \left(\frac{r-1}{r}\right) + 2 \times \frac{1}{r^2} \times \left(\frac{r^2-1}{r^2}\right) + \cdots \\
&\quad + (m-1) \times \frac{1}{r^{(m-1)}} \times \left(\frac{r^{(m-1)}-1}{r^{(m-1)}}\right) \\
&\quad + m \times \frac{1}{r^m} \times \left(\frac{r^m-1}{r^m}\right) \\
&\leq 1 \times \frac{1}{r} + 2 \times \frac{1}{r^2} + \cdots \\
&\quad + (m-1) \times \frac{1}{r^{(m-1)}} + m \times \frac{1}{r^m} \quad (4) \\
\Rightarrow \frac{1}{r} A_m &\leq \frac{1}{r^2} + 2 \times \frac{1}{r^3} + \cdots \\
&\quad + (m-1) \times \frac{1}{r^m} + m \times \frac{1}{r^{(m+1)}} \quad (5)
\end{aligned}$$

$(4) - (5)$

$$A_m - \frac{1}{r} A_m \leq \frac{1}{r} + \frac{1}{r^2} + \cdots$$

$$
\begin{aligned}
& & & +\frac{1}{r^m} - \left(m \times \frac{1}{r^{(m+1)}}\right) \\
\left(1-\frac{1}{r}\right) A_m &\leq& & \frac{1}{r} + \frac{1}{r^2} + \cdots \\
& & & +\frac{1}{r^m} - \left(m \times \frac{1}{r^{(m+1)}}\right) \\
& &=& \frac{\frac{1}{r}\left(1-\frac{1}{r^{(m+1)}}\right)}{1-\frac{1}{r}} - \left(m \times \frac{1}{r^{(m+1)}}\right) \\
& &\leq& \frac{\frac{1}{r}}{1-\frac{1}{r}} \ .
\end{aligned}
$$

Hence, we have

$$
\left(1-\frac{1}{r}\right) A_m \leq \frac{\frac{1}{r}}{1-\frac{1}{r}} \ .
$$

We conclude that

$$
\begin{aligned}
\left(1-\frac{1}{r}\right) A_m &\leq \frac{1}{r-1} \\
\Rightarrow A_m &\leq \frac{r}{(r-1)^2} \ .
\end{aligned}
$$

Q. E. D.

Next, we calculate $\overline{C}_{|LSP(W,P)|,m}$.

Lemma 2 *The value $\overline{C}_{|LSP(W,P)|,m}$ is bounded by $\left(\frac{2r-1}{(r-1)^2}\right)$.*

Proof. By definiton, A_{m-1} denotes that it has $m-1$ new characters in the shift, A_{m-2} denotes that it has $m-2$ new characters in the shift and so on. We have

$$
\begin{aligned}
A_{m-1} &\leq 1 \times \frac{1}{r} + 2 \times \frac{1}{r^2} + \cdots \\
&\quad + (m-1) \times \frac{1}{r^{(m-1)}} + m \times \frac{1}{r^{(m-1)}}, \\
A_{m-2} &\leq 1 \times \frac{1}{r} + 2 \times \frac{1}{r^2} + \cdots \\
&\quad + (m-2) \times \frac{1}{r^{(m-2)}} + m \times \frac{1}{r^{(m-2)}}, \\
&\vdots \\
A_{m-k} &\leq 1 \times \frac{1}{r} + 2 \times \frac{1}{r^2} + \cdots \\
&\quad + (m-k) \times \frac{1}{r^{(m-k)}} + m \times \frac{1}{r^{(m-k)}}, \\
&\leq \frac{r}{(r-1)^2} + \frac{m}{r^{m-k}} \quad \text{for } 1 \leq k \leq m-1
\end{aligned}
$$

where $1 \times \frac{1}{r} + 2 \times \frac{1}{r^2} + \cdots + (m-1) \times \frac{1}{r^{(m-1)}} \leq \frac{r}{(r-1)^2}$. We have

$$
\begin{aligned}
A_{m-1} &\leq \frac{r}{(r-1)^2} + m \times \frac{1}{r^{(m-1)}}, \\
A_{m-2} &\leq \frac{r}{(r-1)^2} + m \times \frac{1}{r^{(m-2)}}, \\
&\vdots \\
A_1 &\leq \frac{r}{(r-1)^2} + m \times \frac{1}{r} \ .
\end{aligned}
$$

We can deduce that

$$
A_m \leq A_{m-1} \leq \cdots \leq A_1 \ . \tag{6}
$$

Note that $\overline{C}_{|W|,m} = \Pr(L=m) \times A_m + \Pr(L=m-1) \times A_{m-1} + \cdots + \Pr(L=1) \times A_1$. By Chebyshev's monotonic inequality [5],

$$
\left(\sum_{k=1}^{n} a_k\right)\left(\sum_{k=1}^{n} b_k\right) \geq n \times \sum_{k=1}^{n} a_k b_k
$$

if $a_1 \leq a_2 \leq \cdots \leq a_n$ and $b_1 \geq b_2 \geq \cdots \geq b_n$. Then,

$$
\begin{aligned}
\left(\sum_{\ell=1}^{m} \Pr(L_j = \ell)\right)\left(\sum_{\ell=1}^{m} A_\ell\right) &\geq m \times \sum_{\ell=1}^{m} \Pr(L_j = \ell) \times A_\ell \\
&= m \times \overline{C}_{|LSP(W,P)|,m}
\end{aligned}
$$

Hence

$$
\begin{aligned}
\overline{C}_{|LSP(W,P)|,m} &\leq \frac{1}{m}\left(\sum_{\ell=1}^{m} \Pr(L_j = \ell)\right)\left(\sum_{\ell=1}^{m} A_\ell\right) \\
&= \frac{1}{m}\left(\sum_{\ell=1}^{m} A_\ell\right) \quad \text{since } \left(\sum_{\ell=1}^{m} \Pr(L_j = \ell)\right) = 1 \\
&\leq \frac{1}{m} \times \left(\frac{r}{(r-1)^2}\right) \\
&\quad + \frac{1}{m} \times \left(\frac{r}{(r-1)^2} + m \times \frac{1}{r^{(m-1)}}\right) + \cdots \\
&\quad + \frac{1}{m} \times \left(\frac{r}{(r-1)^2} + m \times \frac{1}{r}\right) \\
&= \left(\frac{r}{(r-1)^2}\right) \\
&\quad + \frac{1}{m} \times \left(m \times \left(\frac{1}{r^{(m-1)}} + \frac{1}{r^{(m-2)}} + \cdots + \frac{1}{r}\right)\right) \\
&\leq \left(\frac{r}{(r-1)^2}\right) + \frac{1}{m} \times \left(m \times \left(\frac{\frac{1}{r}}{1-\frac{1}{r}}\right)\right) \\
&= \left(\frac{r}{(r-1)^2}\right) + \left(\frac{1}{r-1}\right) \ .
\end{aligned}
$$

Finally, we have

$$
\overline{C}_{|LSP(W,P)|,m} \leq \left(\frac{2r-1}{(r-1)^2}\right).
$$

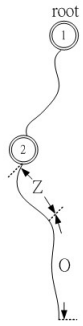

Figure 3: The continuation of character comparisons after $LSP(W,P)$ is obtained.

Q. E. D.

From the above proof, we can see that $\overline{C}_{|LSP(W,P)|,m}$ is very small. But, we still have one problem, that is, after $LSP(W,P)$ is obtained, we may perform several character comparisons afterwards in order to make sure that the suffix-prefix match is the longest. In Fig. 3, the $LSP(W,P)$ is the reverse of the substring from root to node 2. The substring Z denotes the substring which our algorithm continues to traverse and the substring O denotes the substring which is not traversed. The length of Z is of course different for different cases. We shall try to find out the expected length of Z. If it is big, we expect the number of comparisons to be large and if it is small, we would expect the number of comparisons to be small.

Lemma 3 *The expected number of $|Z|$ is bounded by $O(\log_r m)$.*

Proof. Assume that $\overline{C}_{|LSP(W,P)|,m}$ is considered by Lemma 2. Then we have

$$\overline{C}_{|W|,m} = \overline{C}_{|LSP(W,P)|,m} + Z \tag{7}$$

We first note that the

$$\Pr(\text{The length of traversing} > |Z|) \leq \frac{1}{r^{|Z|}}.$$

Thus we actually can somehow know that the chance we will continue searching for a rather lengthy Z is indeed very small.

Let D denote the expected number of character comparisons of the case where the traversing does not reach O. That is, the traversing only traverses a substring Z' where $|Z'| \leq |Z|$. Let E denote the expected number of character comparisons of the case where the entire ZO is traversed. Then, we have

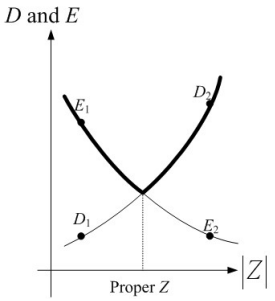

Figure 4: Functions D and E with respect to $|Z|$.

$$D \geq \left(\frac{2r-1}{(r-1)^2} + |Z'|\right)\left(1 - \frac{1}{r^{|Z|}}\right) \tag{8}$$

and

$$\begin{aligned} E &\leq \left(\frac{2r-1}{(r-1)^2} + |Z'| + |O'|\right)\left(\frac{1}{r^{|Z|}}\right) \\ &\leq m\frac{1}{r^{|Z|}} \quad \text{where } |O'| \leq |O|. \end{aligned} \tag{9}$$

It can be easily seen that D increases with $|Z|$ and E decreases with $|Z|$ as shown in Fig. 4. We have

$$\begin{aligned} \overline{C}_{|W|,m} &= \overline{C}_{|LSP(W,P)|,m} + Z \\ &= D + E \\ &\leq \left(\frac{2r-1}{(r-1)^2} + |Z'|\right)\left(1 - \frac{1}{r^{|Z|}}\right) \\ &\quad + m\frac{1}{r^{|Z|}} \ . \end{aligned}$$

Note that both D and E are functions of m and $|Z|$. Our final result of $\overline{C}_{|W|,m}$ cannot be a function of $|Z|$. It can only be a function of m. We have the situation that

$$\overline{C}_{|W|,m} = O(\max(D(m,|Z|), E(m,|Z|))) \quad \text{for all } |Z|$$

We know any $|Z|$ would provide an upper bound for $\overline{C}_{|W|,m}$. However, we want to obtain a better upper bound (i.e., as small as possible in our analysis). Hence, our strategy is to find a proper $|Z|$ that can minimize the order (with respect to m) of $\max(D(m,|Z|), E(m,|Z|))$. Furthermore, we know that the orders of $D(m,|Z|)$ increasing as $|Z|$ increasing and the orders of $E(m,|Z|)$ decreasing as $|Z|$ increasing such that

Table 1: The experimental results for random texts and patterns.

Data source	The length of each string		The alphabet size r	The number total matched comparisons	The number of windows	The theoretical bound $\overline{C}_{\lvert LSP(W,P)\rvert,m}$	The number of average comparisons per window
	The text	The pattern					
Random	10000	30	4	151	338	0.7777	0.446746
	100000	30	4	1316	3377	0.7777	0.389695
	1000000	30	4	13074	33769	0.7777	0.38716
	10000	50	5	64	201	0.5625	0.318408
	100000	50	5	645	2012	0.5625	0.210577
	1000000	50	5	6045	20120	0.5625	0.300447
	10000	30	10	30	334	0.2345	0.08982
	100000	30	10	346	3344	0.2345	0.103469
	1000000	30	10	3930	33463	0.2345	0.117443
	10000	100	7	14	100	0.3611	0.14
	100000	100	7	195	1001	0.3611	0.194805
	1000000	100	7	1865	10018	0.3611	0.186165

$E(m,|Z|) \geq D(m,|Z|)$ when $|Z|$ is small and $E(m,|Z|) \leq D(m,|Z|)$ when $|Z|$ is large. Thus the best choice of $|Z|$ that can minimize the order of $\max(D(m,|Z|), E(m,|Z|))$ can be obtained by solving the equation :

$$D(m,|Z|) = E(m,|Z|) \ .$$

This particular $|Z|$ can be found from Equations (8) and (9).

$$\left(\frac{2r-1}{(r-1)^2}+|Z|\right)\left(1-\frac{1}{r^{|Z|}}\right) = m\frac{1}{r^{|Z|}}$$

$$\Rightarrow \left(\frac{2r-1}{(r-1)^2}+|Z|\right) \leq m\frac{1}{r^{|Z|}} \ .$$

Since $\frac{2r-1}{(r-1)^2}$ is a constant, we have

$$\begin{aligned} |Z| &\leq m\frac{1}{r^{|Z|}} \\ |Z|\, r^{|Z|} &\leq m \\ \log_r |Z| + \log_r r^{|Z|} &\leq \log_r m \\ |Z| &\leq \log_r m \ . \end{aligned}$$

Finally, we have

$$|Z| = O(\log_r m) \ .$$

Q. E. D.

By Equation (7), we have

$$\begin{aligned} \overline{C}_{|W|,m} &= \overline{C}_{|LSP(W,P)|,m} + Z \\ &= \frac{2r-1}{(r-1)^2} + O(\log_r m) \\ &= O(\log_r m) \end{aligned}$$

Lemma 4 *The expected number of string comparisons is based on the longest of suffix of W in T which is equal to a prefix of P to search with a pattern of length m in a text of length n is $O\left(\frac{n\log_r m}{m}\right)$.*

Proof. Each time, on the average, we perform $\overline{C}_{|W|,m}$ comparisons for each window. The average number of new symbols(the region that is not overlapped with the previous window) in a window is $m - \overline{C}_{|LSP(W,P)|,m}$. We share the cost to perform $\overline{C}_{|W|,m}$ characters comparisons evenly to these new symbols. Thus, we get the amortized cost for each character in the text as

$$\frac{\overline{C}_{|W|,m}}{m - \overline{C}_{|LSP(W,P)|,m}} \ .$$

Since there are n characters in the text T, the expectation for the number of character comparisons in the RF algorithm is

$$\begin{aligned} \overline{C}_{n,m} &= n \times \frac{\overline{C}_{|W|,m}}{m - \overline{C}_{|LSP(W,P)|,m}} \\ &= n \times \frac{\log_r m}{m - \frac{2r-1}{(r-1)^2}} \ . \end{aligned}$$

Note that r is the alphabet size of P which is a constant. Therefore, we have the number

$$O\left(\frac{n\log_r m}{m}\right) \ .$$

Q. E. D.

Table 2: The experimental result for human chromosome.

Data source	The length of each string		The alphabet size r	The number total matched comparisons	The number of windows	The theoretical bound $\overline{C}_{\lvert LSP(W,P)\rvert,m}$	The number of average comparisons per window
	The text	The pattern					
Chromosome 21 NT_011512.10	1627105	70	4	8942	23372	0.7777	0.3826
Chromosome 22 NT_011515.11	3437231	70	4	24648	49455	0.7777	0.4984
Chromosome X NT_033330.7	754004	70	4	5029	10843	0.7777	0.4638

4 Experimental Results

We write a program to support the correctness of our proof. We conduct two sets of experiments. In the first set, we randomly generated texts and patterns using by Knuth's random-number-generator function [7] (since the default C random-number generator has some bias). The experimental results are shown in Table 1. In the set, we took three fragments from the human chromosome as T. The pattern is chosen from part of T. The experimental results are shown in Table 2.

We also calculate the length distribution of the longest suffix of a window which is equal to a prefix of the pattern in the above experiments. The result is shown in Table 3. We find that almost all $|LSP\,(W,P)|$ are smaller than 5. Therefore, we conclude that the probability of finding a large $LSP\,(W,P)$ is very small.

5 Conclusion

In this paper, we proposed a simple average-time analysis of the RF algorithm and conducted two sets of experiments to support the theoretical bounds. Both approaches reveals the fact that the RF algorithm is a sub-linear time algorithm in the average case, and it is especially advantageous when the size of the pattern is relatively large.

References

[1] G, Barth, "An analytical comparison of two string searching algorithms," *Information Processing Letters*, Vol. 18, pp. 249-256, 1984.

[2] R. A. Baeza-Yates, "String searching algorithms revisited," *Proceedings of the Workshop on Algorithms and Data Structures*, pp. 75-96, 1989.

[3] R. Boyer, and S. Moore, "A fast string searching algorithm," *Communications of the ACM*, Vol. 20, pp. 762-772, 1977.

[4] M. Crochemore, A. Czumaj, L. Gasieniec, S. Jarominek, T. Lecroq, W. Plandowski and Rytter W. "Speeding up two string-matching algorithms," *Algorithmica*, Vol. 12, pp. 247-267, 1994.

[5] P. L. Čebyšev, "Sue les expressions approximatives des intégrales définies par les autres prises entre les mêmes limites," *Proc. Charkov Math. Soc.*, Vol. 2, pp. 93-98, 1882.

[6] R. N. Horspool, "Practical fast searching in strings," *Software - Practice & Experience*, Vol. 10, pp. 501-506, 1980.

[7] D. E. Knuth, *Art of Computer Programming, Volume 2: Seminumerical Algorithms,* Addison-Wesley, 1997.

[8] D. E. Knuth, J. Morris and V. Pratt, "Fast pattern matching in string," *SIAM Journal on Computing*, Vol. 6, pp. 323-350, 1977.

[9] G. Navarro and M. Raffinot, "A bit-parallel approach to suffix automata: Fast extended string matching," *Proceedings of the 9th Annual Symposium on Combinatorial Pattern Matching*, pp. 14-33, 1998.

[10] A. C. Yao, "The complexity of pattern matching for a random string," *SIAM Journal on Computing*, Vol. 8, pp. 368-387, 1979.

Table 3: The length distribution of the longest suffix of a window which is equal to a prefix of the pattern in these experiments

Data source	The length of each string		The alphabet size r	The number of total matched comparisons	The number of windows	The length distribution of the longest suffix of a window which is equal to a prefix of the pattern												
	The text	The pattern				0	1	2	3	4	5	6	7	8	9	10	30	70
Random	10000	30	4	151	338	240	60	27	7	4	0	0	0	0	0	0	0	0
	100000	30	4	1316	3377	2429	661	225	48	9	5	0	0	0	0	0	0	0
	1000000	30	4	13074	33769	24650	6200	2147	587	128	41	11	4	1	0	0	0	0
	10000	50	5	64	201	150	40	9	2	0	0	0	0	0	0	0	0	0
	100000	50	5	645	2012	1508	395	88	15	1	5	0	0	0	0	0	0	0
	1000000	50	5	6045	20120	15314	3799	812	163	27	5	0	0	0	0	0	0	0
	10000	30	10	30	334	305	28	1	0	0	0	0	0	0	0	0	0	0
	100000	30	10	346	3344	3027	289	27	1	0	0	0	0	0	0	0	0	0
	1000000	30	10	3930	33463	29959	3119	353	27	6	0	0	0	0	0	0	0	0
	10000	100	7	14	100	86	14	0	0	0	0	0	0	0	0	0	0	0
	100000	100	7	195	1001	841	131	23	6	0	0	0	0	0	0	0	0	0
	1000000	100	7	1939	10019	8475	1276	219	42	5	1	0	0	0	0	0	0	0
Chromosome 21 NT_011512.10	1627105	70	4	8942	23372	16265	5823	967	213	68	16	13	3	2	1	0	0	1
Chromosome 22 NT_011515.11	3437231	70	4	24648	49455	32269	11843	3990	891	295	111	45	6	2	1	1	0	1
Chromosome X NT_033330.7	754004	70	4	5029	10843	7177	2722	701	164	54	17	7	0	0	0	0	0	1